INGENIERÍA DEL SOFTWARE • CÁTEDRA DE CALIDAD DE SOFTWARE

Técnicas y Niveles de Pruebas de Software

Guía de Exposición Grupal y Manual Técnico: Fundamentos, Pirámide de Pruebas, Backend, Integración, Sistema, Interfaz de Usuario (GUI) y Pruebas de Humo.

ESTRUCTURA
4 Puntos de Exposición (1 por Integrante)

TECNOLOGÍAS PRÁCTICAS
PHP (PHPUnit) & Python (pytest)

REFERENCIAS PRINCIPALES
Pressman, Fowler, Jorgensen, McConnell, Kaner

Distribución del Equipo: 4 Puntos de Exposición

Asignación equitativa para 4 expositores

PUNTO 1 • EXPOSITOR 1

Fundamentos, Pirámide y Pruebas Unitarias

Introducción metodológica, pirámide clásica y moderna, patrón AAA, dobles de prueba (Mocks vs Stubs), cobertura de código y ejemplos prácticos en PHPUnit y pytest.

PUNTO 2 • EXPOSITOR 2

Pruebas de Backend (Puente Niveles 1 y 2)

Definición según Martin Fowler, ámbito del servidor sin GUI, endpoints REST, validación transaccional de BD, diferencia clave con Sistema y automatización con Postman/PHPUnit/pytest.

PUNTO 3 • EXPOSITOR 3

Pruebas de Integración y Estrategias

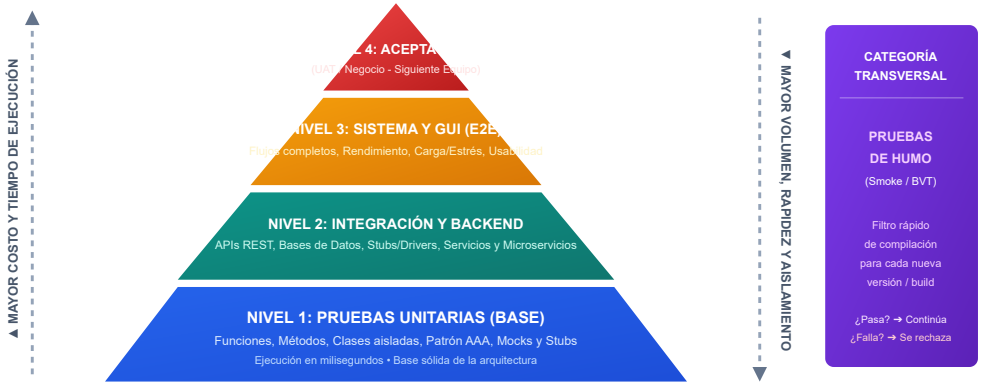
Definición de Pressman, diferenciación conceptual Stubs vs Drivers, las 4 estrategias (Big-Bang, Top-Down, Bottom-Up, Híbrida) y pruebas de persistencia e interfaces.

PUNTO 4 • EXPOSITOR 4

Sistema, Interfaz (GUI) y Pruebas de Humo

Pruebas de Sistema (SRS), tabla Carga vs Estrés, automatización GUI con Page Object Model (Playwright/Cypress), y Pruebas de Humo transversales (McConnell / BVT vs Regresión).

DIAGRAMA OFICIAL: PIRÁMIDE DE NIVELES DE PRUEBAS Y EJE TRANSVERSAL



PUNTO 1: Fundamentos, Pirámide y Pruebas Unitarias

EXPOSITOR 1

1.1 Introducción y la Estrategia de Niveles de Pruebas

En la ingeniería del software, las pruebas no son un evento final y aislado, sino un proceso riguroso de verificación y validación que acompaña todo el ciclo de vida del desarrollo. Ninguna técnica individual puede descubrir todos los defectos; por ello, se adopta un enfoque estratificado en niveles.

> **Roger S. Pressman:** "La estrategia clásica de pruebas de software se organiza en una progresión sistemática: pruebas de unidad, de integración, de sistema y de aceptación. El proceso avanza desde los componentes individuales más pequeños hacia el ensamblaje del sistema global."
>
> *— Ingeniería del Software: Un enfoque práctico, 7ma Edición.*

La **Pirámide de Pruebas** (popularizada por Mike Cohn y formalizada por Martin Fowler) postula que la base de la inversión en aseguramiento de calidad debe sustentarse en un volumen masivo de pruebas unitarias automáticas, rápidas y de bajo costo. A medida que se asciende hacia integración y pruebas de sistema/GUI, el número de pruebas disminuye debido a su mayor lentitud, fragilidad y alto costo de mantenimiento.

1.2 Pruebas Unitarias o de Unidad (Nivel 1 - Base)

> **Definición Canónica (Pressman, Pág. 389):** "Las pruebas unitarias enfocan la verificación en la unidad más pequeña del diseño del software: un componente o módulo individual. Su propósito primordial es detectar errores dentro de ese componente examinando su lógica algorítmica interna y sus estructuras de datos locales de forma totalmente aislada."

Características esenciales de una prueba unitaria de alta calidad:

- **Rápida:** Se ejecuta en milisegundos; una suite de miles de pruebas unitarias debe culminar en pocos segundos.
- **Aislada e Independiente:** No interactúa con recursos externos volátiles (bases de datos de producción, red, disco ni servicios web).
- **Repetible y Determinista:** Produce idéntico resultado en cualquier máquina y momento, sin importar el orden de ejecución.
- **Autovalidada:** Determina automáticamente si el caso pasó o falló mediante aserciones formales, sin inspección manual humana.

1.3 El Patrón AAA (Arrange - Act - Assert)

Es el estándar de la industria para estructurar casos de prueba legibles, modulares y mantenibles:

1. Arrange (Organizar / Preparar)

Se inicializan los objetos bajo prueba, se definen los parámetros de entrada y se configuran los dobles de prueba (Mocks o Stubs) necesarios.

2. Act (Actuar / Ejecutar)

Se invoca la función o método específico que se desea validar con las entradas preparadas.

3. Assert (Afirmar / Verificar)

Se comprueba mediante aserciones que la salida real obtenida coincida exactamente con el valor o estado esperado, o que se lance la excepción correcta.

1.4 Conceptos Críticos: Mocks vs. Stubs y Cobertura de Código

Stubs (Resguardos Pasivos)

Objetos que devuelven respuestas predefinidas enlatadas para alimentar la prueba. No verifican comportamiento; solo suministran datos para que el método avance sin fallar.

Mocks (Simuladores con Verificación)

Objetos con expectativas activas. Registran llamadas y permiten afirmar si un método dependiente fue ejecutado con argumentos exactos y una cantidad específica de veces.

Cobertura de Código (Code Coverage): Métrica cuantitativa que indica el porcentaje de líneas, ramas (branch coverage) o rutas ejecutadas por la suite de pruebas. Un umbral saludable oscila entre el **75% y 85%** en reglas de negocio críticas. *Importante para la exposición:* 100% de cobertura no certifica ausencia de defectos (se puede ejecutar código sin evaluar aserciones significativas); sin embargo, una cobertura nula garantiza vulnerabilidad e incertidumbre técnica.

1.5 Ejemplos Prácticos de Pruebas Unitarias

tests/Unit/CalculadoraDescuentoTest.php — PHP • PHPUNIT

```php
<?php
use PHPUnit\Framework\TestCase;

class CalculadoraDescuentoTest extends TestCase {
    public function testCalculaTotalConDescuentoEImpuesto(): void {
        // 1. ARRANGE (Organizar): Crear Mock de la tasa de impuesto y la instancia
        $mockImpuesto = $this->createMock(ServicioImpuestoInterface::class);
        $mockImpuesto->method('obtenerTasa')->willReturn(0.16); // 16% IVA

        $calculadora = new CalculadoraDescuento($mockImpuesto);

        // 2. ACT (Actuar): Subtotal $100 con cupón PROMO10 (10% descuento = $90 base)
        $total = $calculadora->calcular(100.0, 'PROMO10');

        // 3. ASSERT (Afirmar): $90 + 16% IVA ($14.4) = $104.4
        $this->assertEquals(104.4, $total, "El total con descuento e impuesto debe ser 104.4");
    }

    public function testLanzaExcepcionSiMontoEsNegativo(): void {
        $mockImpuesto = $this->createStub(ServicioImpuestoInterface::class);
        $calculadora = new CalculadoraDescuento($mockImpuesto);

        $this->expectException(\InvalidArgumentException::class);
        $calculadora->calcular(-50.0, 'PROMO10');
    }
}
```

tests/test_calculadora.py PYTHON • PYTEST

```python
import pytest
from unittest.mock import Mock
from app.servicios import calcular_total_pedido

# Fixture reutilizable para preparar el servicio simulado
@pytest.fixture
def mock_impuesto():
    servicio = Mock()
    servicio.obtener_tasa.return_value = 0.16  # Stub de 16%
    return servicio

def test_calculo_total_con_descuento_valido(mock_impuesto):
    # Arrange
    subtotal = 100.0
    cupon = "PROMO10"

    # Act
    resultado = calcular_total_pedido(subtotal, cupon, mock_impuesto)

    # Assert
    assert resultado == 104.4
    mock_impuesto.obtener_tasa.assert_called_once()  # Verificación de Mock

def test_monto_negativo_lanza_error(mock_impuesto):
    with pytest.raises(ValueError, match="El monto no puede ser negativo"):
        calcular_total_pedido(-25.0, "PROMO10", mock_impuesto)
```

TIP DE EXPOSICIÓN PARA EL INTEGRANTE 1:

Destaca ante el profesor que las pruebas unitarias **nunca deben conectarse a la base de datos real ni a internet**; para eso existen los dobles de prueba (Mocks y Stubs). Si se conectan al exterior, dejan de ser unitarias y se transforman en pruebas de integración.

PUNTO 2: Pruebas de Backend (El Puente entre Niveles 1 y 2)

EXPOSITOR 2

2.1 Definición y Alcance según la Ingeniería del Software

> **Martin Fowler:** "Las evaluaciones focalizadas en servicios o API examinan la lógica operativa, la persistencia en bases de datos y los esquemas de seguridad sin interactuar con la capa visual. Representan una verificación rigurosa de caja negra en la infraestructura del servidor."
>
> *— Testing Strategies in a Microservice Architecture.*

ACLARACIÓN ESTRUCTURAL CLAVE: BACKEND COMO PUENTE DE NIVELES 1 Y 2

Las pruebas de Backend unen conceptualmente el **Nivel 1 (Unitario)** y el **Nivel 2 (Integración)**. Dentro del backend se prueban tanto componentes aislados (controladores, servicios internos y entidades sin HTTP ni GUI) como la integración directa entre endpoints HTTP, serialización JSON, reglas transaccionales ACID y bases de datos relacionales o NoSQL.

2.2 Diferencia Crucial: Pruebas de Backend vs. Pruebas de Sistema

Es un error habitual confundir las pruebas de backend con las de sistema. La distinción es nítida:

CRITERIO	PRUEBAS DE BACKEND (SERVICIOS / APIS)	PRUEBAS DE SISTEMA (SYSTEM TESTING)
Ámbito	Capa del servidor: controladores, microservicios, APIs REST y BD.	Todo el sistema integrado: Frontend + Backend + Hardware + Red.
Capa Visual	Completamente excluida (se prueba vía HTTP directo o llamadas RPC).	Incluida en los flujos completos de usuario (E2E).
Objetivo	Validar lógica de negocio, esquemas de datos, contratos API y BD.	Validar cumplimiento global de la especificación SRS del cliente.
Velocidad	Rápida (sin renderizado visual en navegador web).	Lenta (espera cargas de assets, renderizado y red completa).

2.3 Validación de Base de Datos en el Servidor

Una faceta central de las pruebas de backend es asegurar la integridad del modelo de persistencia:

- **Transacciones ACID:** Verificar que operaciones con fallos intermedios apliquen *Rollback* estricto y no corrompan los saldos o inventarios.
- **Restricciones de Integridad:** Comprobar que se respeten claves foráneas (FK), índices únicos y restricciones NOT NULL.
- **Procedimientos Almacenados y Triggers:** Validar que los eventos automáticos de base de datos disparen los cálculos esperados ante inserciones o bajas.

2.4 Flujo de Trabajo Metódico en 5 Pasos

1. **1. Aprovisionamiento:** Configurar el entorno con base de datos de pruebas aislada y fixtures de datos conocidas.
2. **2. Diseño de Casos:** Estructurar escenarios de contratos API (códigos 200/201 para éxito, 400 para validación, 401/403 para autenticación).
3. **3. Ejecución Automatizada:** Lanzar peticiones HTTP o llamadas de servicio mediante herramientas CLI o de integración.
4. **4. Verificación de Estado:** Comprobar códigos HTTP, cuerpo JSON devuelto y la persistencia física en la base de datos.
5. **5. Registro y CI/CD:** Consolidar reportes de fallos de endpoints e incorporar la suite a la canalización de integración continua.

2.5 Ejemplos Prácticos de Pruebas de Backend (APIs y Servicios)

tests/Backend/ApiArticulosTest.php — PHP • PHPUNIT / CONTROLLER TEST

```php
<?php
use PHPUnit\Framework\TestCase;

class ApiArticulosTest extends TestCase {
    public function testCrearArticuloRetornaHttp201YJsonValido(): void {
        $controlador = new ControladorArticulos();
        $payload = [
            'codigo' => 'ART-2026',
            'nombre' => 'Monitor 27 Pulgadas 144Hz',
            'precio' => 249.99,
            'stock'  => 15
        ];

        // Ejecución en backend: Procesa lógica y guarda en BD
        $respuesta = $controlador->guardarArticulo($payload);

        // Validaciones de API Backend
        $this->assertEquals(201, $respuesta->getStatusCode());
        $this->assertTrue($respuesta->hasHeader('Content-Type', 'application/json'));

        $cuerpo = json_decode($respuesta->getBody(), true);
        $this->assertEquals('success', $cuerpo['estado']);
        $this->assertArrayHasKey('id_generado', $cuerpo);
    }
}
```

tests/backend/test_api_backend.py — PYTHON • PYTEST + TESTCLIENT

```python
import pytest
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_registro_articulo_backend_exitoso():
    payload = {
        "sku": "LAP-001",
        "nombre": "Laptop ThinkPad",
        "precio": 850.00,
        "categoria_id": 3
    }
    response = client.post("/api/v1/articulos", json=payload)

    # Aserciones de API Backend
    assert response.status_code == 201
    data = response.json()
    assert data["sku"] == "LAP-001"
    assert data["activo"] is True

def test_validacion_error_422_por_precio_invalido():
    payload = {"sku": "LAP-001", "precio": -10.0}
    response = client.post("/api/v1/articulos", json=payload)
    assert response.status_code == 422  # Unprocessable Entity
```

Herramientas destacadas: **Postman** (diseño y colecciones automatizadas con Newman), **Apache JMeter** (rendimiento y estrés de endpoints), **sqlmap** (auditoría de inyección SQL) y frameworks de código como **PHPUnit** y **pytest**.

PUNTO 3: Pruebas de Integración (Nivel 2: Interfaces y Estrategias) EXPOSITOR 3

3.1 Definición y Enfoque Metodológico

FRASE PUENTE PARA LA EXPOSICIÓN

"Como acabamos de ver con las pruebas de backend, los componentes del servidor necesitan comunicarse continuamente entre sí. Ahora profundizamos formalmente en el **Nivel 2 de la Pirámide: Pruebas de Integración**, analizando cómo validar la interacción entre múltiples módulos ensamblados."

> **Roger S. Pressman:** "Las pruebas de integración son una técnica sistemática para construir la arquitectura del software y, al mismo tiempo, realizar pruebas que descubran errores asociados con las interfaces entre los componentes. Se unen módulos previamente probados de forma individual y se comprueba que colaboren correctamente."
>
> *— Software Engineering: A Practitioner's Approach.*

3.2 Diferenciación Fundamental: Stubs vs. Drivers

En integración, casi nunca todos los componentes están listos al mismo tiempo. Para probar módulos intermedios se utilizan dos tipos de sustitutos con funciones opuestas:

Drivers (Controladores / Módulos Conductores)

Ubicación: Capa Superior (Llamador).

Un Driver es un programa auxiliar que *invoca al módulo bajo prueba*, pasándole parámetros y recibiendo resultados. Se necesita cuando los módulos superiores de coordinación aún no han sido desarrollados.

Stubs (Resguardos / Servidores Falsos)

Ubicación: Capa Inferior (Llamado).

Un Stub es un módulo simulado que *es invocado por el módulo bajo prueba*. Responde con datos sintéticos para sustituir servicios o bases de datos de bajo nivel que aún no están disponibles o son inestables.

ESQUEMA ARQUITECTÓNICO: USO DE DRIVERS Y STUBS EN INTEGRACIÓN



3.3 Las Cuatro Estrategias de Integración

ESTRATEGIA	MECANISMO DE ENSAMBLAJE	ELEMENTOS NECESARIOS	DIFICULTAD DE DEPURACIÓN
1. Big-Bang	Todos los módulos se unen y se prueban en un solo paso masivo.	Ninguno (solo el sistema listo).	**Extrema:** Imposible aislar qué módulo originó la falla.
2. Ascendente (Bottom-Up)	Inicia en los módulos más bajos (acceso a BD, utilidades) y sube progresivamente.	Requiere **Drivers** para emular los niveles superiores.	**Baja:** Los cimientos se estabilizan tempranamente.
3. Descendente (Top-Down)	Inicia en los módulos arquitectónicos y de control superior y desciende hacia las hojas.	Requiere **Stubs** para simular las funciones inferiores faltantes.	**Moderada:** Valida reglas de negocio principales desde el inicio.
4. Híbrida / Sandwich	Combina Bottom-Up en capas de infraestructura y Top-Down en capas de negocio.	Utiliza simultáneamente **Drivers y Stubs**.	**Controlada:** La más balanceada en proyectos industriales.

3.4 Ejemplos Prácticos de Pruebas de Integración

tests/Integration/ServicioInventarioDbTest.php — PHP • PHPUNIT CON BASE DE DATOS SQLITE

```php
<?php
use PHPUnit\Framework\TestCase;

class ServicioInventarioDbTest extends TestCase {
    private \PDO $pdo;

    protected function setUp(): void {
        // Integración real contra SQLite en memoria (aislado pero con motor SQL real)
        $this->pdo = new \PDO('sqlite::memory:');
        $this->pdo->exec("CREATE TABLE articulos (id INTEGER PRIMARY KEY, sku TEXT, stock INT)");
    }

    public function testIntegracionServicioYRepositorioPersistenEnBaseDatos(): void {
        $repositorio = new RepositorioArticulosPdo($this->pdo);
        $servicio = new ServicioInventario($repositorio);

        // Se ejecuta el flujo integrado de múltiples clases
        $servicio->ingresarStock('SKU-88', 50);

        // Se comprueba la persistencia real consultando el registro
        $stmt = $this->pdo->query("SELECT stock FROM articulos WHERE sku = 'SKU-88'");
        $fila = $stmt->fetch(\PDO::FETCH_ASSOC);

        $this->assertEquals(50, (int)$fila['stock']);
    }
}
```

tests/integration/test_integracion_servicio_db.py — PYTHON • PYTEST + SQLITE FIXTURE

```python
import pytest
import sqlite3
from app.repositorios import RepositorioUsuario
from app.servicios import ServicioAutenticacion

@pytest.fixture
def db_conexion():
    conn = sqlite3.connect(":memory:")  # Base de datos en memoria para integración
    conn.execute("CREATE TABLE usuarios (id INT, email TEXT, clave_hash TEXT)")
    yield conn
    conn.close()

def test_integracion_servicio_autenticacion_con_bd_real(db_conexion):
    repo = RepositorioUsuario(db_conexion)
    servicio = ServicioAutenticacion(repo)

    # Act: Registrar usuario e intentar login
    servicio.registrar_usuario("admin@empresa.com", "ClaveSegura123!")
    es_valido = servicio.validar_credenciales("admin@empresa.com", "ClaveSegura123!")

    # Assert: Verifica la comunicación real entre servicio de hashing y tabla SQL
    assert es_valido is True
```

PUNTO 4: Pruebas de Sistema, Interfaz (GUI) y Pruebas de Humo

EXPOSITOR 4

4.1 Pruebas de Sistema (Nivel 3)

> **Roger S. Pressman:** "Las pruebas de sistema evalúan el software completamente integrado con los demás elementos del entorno —hardware, personas y bases de datos operativas— para comprobar que el conjunto completo responde a los requisitos globales."
>
> *— Software Engineering: A Practitioner's Approach.*

> **Paul C. Jorgensen:** "La prueba de sistema examina el comportamiento del sistema completo y totalmente integrado contra la Especificación de Requisitos de Software (SRS), validando flujos de trabajo de extremo a extremo bajo un enfoque estricto de caja negra."
>
> *— Software Testing: A Craftsman's Approach, 4ta Edición.*

Tabla Comparativa Fundamental: Pruebas de Carga vs. Pruebas de Estrés

PARÁMETRO	PRUEBAS DE CARGA (LOAD TESTING)	PRUEBAS DE ESTRÉS (STRESS TESTING)
Objetivo	Validar el rendimiento y los tiempos de respuesta bajo el tráfico previsto normal o pico planificado.	Llevar el sistema *más allá* del límite máximo para provocar la falla y evaluar su punto de quiebre.
Volumen	Dentro de los límites esperados según la especificación del negocio.	Cargas extremas, saturación de memoria, desconexión abrupta de servidores.
Resultado Esperado	El sistema responde de manera fluida y dentro del umbral SLA acordado.	Degradación controlada, sin corrupción de datos y con capacidad de auto-recuperación.

Herramientas de Ejecución vs Gestión: Es fundamental diferenciar las herramientas de ejecución real (**Apache JMeter, k6, Locust, OWASP ZAP**) de aquellas que solo sirven para la gestión de casos y reporte de defectos (**TestRail, Jira, Bugzilla**).

4.2 Pruebas de Interfaz Gráfica de Usuario (UI/GUI Testing)

> **Cem Kaner:** "Estas evaluaciones comprueban la correcta renderización visual y operatividad de los componentes con los que interactúa el usuario final, abarcando elementos visuales, botones, formularios y navegación."
>
> *— Testing Computer Software.*

El Patrón Page Object Model (POM): Es el patrón arquitectónico estándar de automatización de GUI. Separa la lógica de prueba de los selectores HTML específicos encapsulándolos en clases representativas de página. Si un botón o campo cambia de selector en el HTML, solo se actualiza una clase Page Object sin alterar cientos de pruebas existentes.

Herramientas Modernas de GUI: Playwright (de Microsoft, ultra rápido y multi-lenguaje), **Cypress** (excelente para Single Page Apps en JavaScript) y **Selenium WebDriver** (el estándar histórico multiplataforma).

tests/gui/test_login_page_object.py

PYTHON • PLAYWRIGHT + PAGE OBJECT MODEL

```python
from playwright.sync_api import Page, expect

class LoginPage:
    def __init__(self, page: Page):
        self.page = page
        self.input_usuario = page.locator("#txt_usuario")
        self.input_clave = page.locator("#txt_password")
        self.btn_ingresar = page.locator("button[type='submit']")

    def iniciar_sesion(self, usuario: str, clave: str):
        self.input_usuario.fill(usuario)
        self.input_clave.fill(clave)
        self.btn_ingresar.click()

def test_login_exitoso(page: Page):
    page.goto("https://sistema2026.local/login")
    login_page = LoginPage(page)

    # Ejecución de la acción de interfaz de usuario
    login_page.iniciar_sesion("operador1", "Secret2026*")

    # Verificación de GUI: Debe redirigir y mostrar el Dashboard
    expect(page).to_have_url("https://sistema2026.local/dashboard")
    expect(page.locator("#bienvenida")).to_contain_text("Bienvenido, operador1")
```

FRASE PUENTE AL NIVEL 4 (CÚSPIDE DE LA PIRÁMIDE)

"El **Nivel 4 corresponde a las Pruebas de Aceptación del Usuario (UAT)**, donde el cliente y los usuarios finales verifican el sistema en base a los criterios de aceptación del negocio para autorizar el paso a producción. Este nivel corresponde desarrollarlo al siguiente equipo expositor."

4.3 Pruebas de Humo (Smoke Testing - Categoría Transversal)

> **Steve McConnell:** "Adaptado de la ingeniería de hardware, este concepto consiste en una Verificación Preliminar de Construcción (Build Verification Testing - BVT). Su meta primordial radica en validar que las funciones críticas y de mayor prioridad operen adecuadamente tras una compilación reciente; si esta prueba falla, la versión es rechazada de inmediato."
>
> *— Rapid Development: Taming Wild Software Schedules.*

FLUJO OPERATIVO DE PRUEBAS DE HUMO (BVT)



Tabla de Diferenciación: Smoke vs. Sanity vs. Regression Testing

TÉCNICA	MOMENTO DE EJECUCIÓN	ALCANCE Y PROFUNDIDAD	OBJETIVO PRIMORDIAL
Smoke Testing	Inmediatamente tras compilar una nueva versión.	**Muy amplio y superficial:** Toca todos los módulos principales sin entrar en detalle.	Comprobar si la versión es suficientemente estable para ser evaluada.
Sanity Testing	Tras aplicar una corrección de error (bugfix) puntual.	**Muy estrecho y profundo:** Se centra estrictamente en la funcionalidad reparada.	Validar que la falla reportada fue solucionada antes de continuar.
Regression Testing	De forma periódica antes de pases a producción.	**Exhaustivo:** Vuelve a ejecutar la suite completa de pruebas funcionales.	Garantizar que nuevos cambios no hayan deteriorado funcionalidades previas.

¿QUÉ ENTRA EN UNA PRUEBA DE HUMO PRÁCTICA?:
Solo flujos esenciales ("Happy Paths"): **1.** Inicio de sesión con usuario válido. **2.** Carga correcta de la vista principal. **3.** Conexión activa a la base de datos. **4.** Creación de una transacción mínima. Tiempo límite: menor a 10 minutos.